not significant. That said, two-character indentation—spaces, not tabs—is the overwhelming choice of the Ruby community.

Methods are defined with the keyword def, followed by the method name—in this case, the name is say_hello_goodbye—and then the method's parameters between parentheses. (In fact, the parentheses are optional, but we recommend you use them.) Ruby doesn't use braces to delimit the bodies of compound statements and definitions. Instead, you simply finish the body with the keyword end. Our method's body is pretty simple. The first line concatenates the literal string "I don't know why you say goodbye, " and the parameter name and the literal string ", I say hello" and assigns the result to the local variable result. The next line returns that result to the caller. Note that we didn't have to declare the variable result; it sprang into existence when we assigned to it.

Having defined the method, we invoke it twice. In both cases, we pass the result to the method puts, which simply outputs its argument followed by a newline (moving on to the next line of output):

```
I don't know why you say goodbye, John, I say hello
I don't know why you say goodbye, Paul, I say hello
```

The line puts say_hello_goodbye("John") actually contains two method calls, one to the method say_hello_goodbye with the argument "John" and the other to the method puts whose argument is the result of the call to say_hello_goodbye . Why does one call have its arguments in parentheses while the other doesn't? In this case, it's purely a matter of taste—the puts method is available to all objects and is often written without parentheses around its argument. Ruby does not require parenthesis unless they are directly needed for the interpreter to parse the statement the way you want. The following lines are equivalent:

```
puts say_hello_goodbye("John")
puts(say_hello_goodbye("John"))
```

Life isn't always simple, and precedence rules can make it difficult to know which argument goes with which method invocation, so we recommend using parentheses in all but the simplest cases. You will see that Ruby programs often omit the parenthesis when the method does not have an explicit receiver and only has one argument.

This example also shows Ruby string objects. Ruby has many ways to create a string object, but the most common is to use *string literals*, which are sequences of characters between single or double quotation marks. The two forms differ in the amount of processing Ruby does on the string while constructing the literal. In the single-quoted case, Ruby does very little. With a few exceptions, what you enter in the string literal becomes the string's value.

In the double-quoted case, Ruby does more work. First, it looks for substitution sequences that start with a backslash character and replaces them with some binary value. The most common of these substitutions is \n, which is replaced with a newline character. When a string containing a newline is output, that newline becomes a line break:

```
puts "Hello and goodbye to you,\nGeorge"
```

produces:

```
Hello and goodbye to you,
George
```